

프로젝트 보고서 — 기말 (MLOps)

- 날짜: 2026-06-20
- 학번: 213255
- 이름: 최영현
- 과목: DevOps / MLOps

중간 프로젝트(반려동물 A6 스티커 생성 웹 서비스)를 기반으로, **스티커 제작 품질 예측 ML 기능**을 추가하고 MLflow 실험관리·모델버전·재학습·롤백·예측/피드백 로깅·운영 모니터링·CI/CD·Docker·배포까지 이어지는 **MLOps 파이프라인**을 구축한 결과를 정리한다.

1. 프로젝트 개요

- **프로젝트 이름:** 멍스티커 (Pet Sticker Generator) + 제작 품질 예측 MLOps
- **프로젝트 목적:** 반려동물 사진을 업로드해 A6 스티커 시안을 생성하는 서비스에, 업로드 사진의 **인쇄 제작 품질을 예측**(점수 0~100 + 제작 적합 / 보정 권장 / 재촬영 권장 3분류 + 개선 추천)하는 ML 기능을 결합한다. 단순 모델 학습이 아니라 학습→등록→서빙→교체→롤백→모니터링→재학습의 **운영 파이프라인**을 구현하는 것이 목표다.
- **GitHub 주소:** <https://github.com/thighburger/StickerGenerator> (public)
- **배포 주소 및 캡처:**
 - ML 추론 서비스(FastAPI) — Render(Docker): <https://pet-sticker-ml.onrender.com> ← 라이브 배포 완료·외부 접속 확인 됨. `render.yaml` 블루프린트로 `main` 브랜치에서 자동 배포(Region: Oregon). `/health` · `/model/info` · `/docs` 정상 응답.
 - 웹 앱(Next.js) — Vercel: `main` 브랜치 자동 배포(빌드 성공). 환경변수 `ML_SERVICE_URL` = 위 Render URL 로 ML 연결.

배포 확인 — 실제 외부 응답:

```
$ curl https://pet-sticker-ml.onrender.com/health
{"status":"ok","modelLoaded":true,"modelVersion":"v1"}
$ curl https://pet-sticker-ml.onrender.com/model/info
{"modelName":"pet-sticker-quality","alias":"champion","version":1,"dataVersion":"v1", ...}
```

Pretty-print ☐

```
{"status": "ok", "modelLoaded": true, "modelVersion": "v1"}
```

Pet Sticker Quality ML

0.1.0 OAS 3.1

/openapi.json

반려동물 스티커 제작 품질 예측 추론 서비스 (champion 모델 서빙)

default



GET	/health	Health	▼
POST	/predict	Predict	▼
POST	/predict/features	Predict Features	▼
POST	/feedback	Feedback	▼
GET	/model/info	Model Info	▼
POST	/model/reload	Reload Model	▼
GET	/logs/summary	Logs Summary	▼

Schemas



Body_predict_predict_post > Expand all object

- MLflow Tracking Server 주소 및 캡처: `http://localhost:5000` (MLflow UI, 파일스토어 `mlruns/` 기반). 실험·run·등록 모델을 이 서버에서 관리한다. 아래 캡처(실험 run 목록·출처·등록 모델). 상세는 9절 참조.

mlflow2.22.5

ExperimentsModelsPrompts

pet-sticker-quality

Provide Feedback

Add Description

Share

Search experiments

pet-sticker-quality

RunsEvaluationExperimentalTraces

metrics.rmse < 1 and params.model = "tree"

Time created

State: Active

+ New run

Datasets

Sort: Created

Columns

Group by

	Run Name	Created	Dataset	Duration	Source	Models
	quality-rf-v2	22 seconds ago	-	3.0s	train.py	pet-sticl
	quality-rf-v1	1 minute ago	-	38.0s	train.py	pet-sticl
	quality-rf-v1	1 hour ago	-	2.1s	train.py	pet-sticl
	quality-rf-v2	1 hour ago	-	13.8s	retrain.py	pet-sticl
	quality-rf-v1	1 hour ago	-	35.3s	train.py	pet-sticl

5 matching runs

캡처 안내: 앱 메인·관리자(로그인 포함)·ML 서비스(라이브 Render)·MLflow·CI 화면 캡처를 모두 docs/assets/ 에 포함했다(각 절 참조).

2. 소프트웨어 주요 기능

ML 기능과 일반 서비스 기능을 분리해 정리한다.

2-1. 사용자 핵심 기능(서비스)

- 반려동물 사진 업로드(최대 5장) → 배경 제거(remove.bg) → 캔버스에서 A6(1240×1748) 시안 자동 생성
- 시안 미리보기 및 고화질 PNG 다운로드
- 주문 생성: 주문번호 발급 → 주문 파일 저장 → 카카오톡 오픈채팅 문의

주문 후 진행 과정

시안 PNG가 저장되고 주문번호가 복사돼요.

열린 채팅방에 주문번호를 붙여
넣어 주세요.

결제 확인 후 스티커 제작을 시작해요.

안전하게 포장하여 빠르게 배송
해 드려요.

- 구매(주문 저장) 시점: Next.js /api/orders 가 업로드된 첫 사진을 ML 서비스 POST /predict 로 전달
- 응답(품질 점수-클래스-추천-모델버전)을 주문 정보(order.json)에 mlReport 로 저장하고 메인 화면에 품질 결과 카드로 표시
- 관리자 화면 (/admin) 에서 주문별 ML 점수-모델버전, 예측/피드백 로그 요약, 채피언 모델 정보 확인

2-3. 입력/출력

구분	입력	출력
서비스	반려동물 사진(최대 5장)	A6 스티커 시안 PNG
ML	사진(이미지 바이트) → 특징 10종 추출	score(0~100) , qualityClass(3분류) , recommendation , confidence , modelVersion

Pet Sticker Quality ML 0.1.0 OAS 3.1 /openapi.json 반려동물 스티커 제작 품질 예측 추론 서비스 (champion 모델 서빙)	
default	
GET	/health Health
POST	/predict Predict
POST	/predict/features Predict Features
POST	/feedback Feedback
GET	/model/info Model Info
POST	/model/reload Reload Model
GET	/logs/summary Logs Summary
Schemas	
Body_predict_predict_post > Expand all object	
FeatureRequest > Expand all object	
FeedbackRequest > Expand all object	

3. 실행 환경

- **OS:** macOS 26 (개발) / Ubuntu(GitHub Actions, Render 컨테이너)
- **Git/GitHub:** GitHub 저장소 + GitHub Actions CI/CD
- **Docker:** pet-sticker-ml/Dockerfile (python:3.12-slim), pet-sticker-next/Dockerfile (node:20-alpine, 멀티스 테이지), 루트 docker-compose.yml
- **MLflow:** 2.22.x — 로컬 파일스토어(mlruns/ , CI/하네스) + mlflow server (sqlite, 보고서 캡처용, <http://localhost:5000>)
- **언어/런타임:** Next.js 14.2 / React 18 / TypeScript 5, Python 3.12 / FastAPI / scikit-learn 1.9 / OpenCV(headless)
- **배포 환경:** Vercel(웹), Render(ML, Docker)

4. 전체 MLOps 파이프라인 구조

```
[코드 변경 흐름] 개발 → Git 커밋 → push → GitHub Actions(CI: next/ml/harness/docker) → Vercel/Render 배포
[모델 학습 흐름] data(csv) → train.py(MLflow: param·metric·artifact·model + 레지스트리 등록)
[등록/반영 흐름] model_promoter(macro-F1 개선 시 @champion 승격) → champion/export(model.pkl+metadata.json)
    → FastAPI 가 champion/ 로드 → /model/reload 로 무중단 반영
[서비스 운영 흐름] Next /api/orders → ML /predict → mlReport 저장 + 예측 로그 → /feedback → 피드백 로그
    → /admin 모니터링 → (피드백 반영) 재학습(v2) → 승격 또는 롤백
```

관리자 대시보드(/admin)는 주문별 ML 점수·모델버전, 주문 통계, 챔피언 모델 정보(라이브 Render), 예측/피드백 로그 요약을 표시한다. 관리자 인증으로 보호되며, 아래는 로그인 후 화면이다.

운영 관리자 대시보드

← 생성기로 로그아웃

주문별 ML 품질 점수 · 챔피언 모델 버전 · 예측/피드백 로그

챔피언 모델

모델

pet-sticker-quality

버전

v1

데이터

v1

macro-F1

0.7549

정확도

0.7567

학습 시각

2026-06-19T16:26:56.005227+00:00

3

주문 수

57.4

평균 품질점수

1

제작 적합

1

보정 권장

1

재활영 권장

주문 목록

샘플 데이터

주문번호	이름	생성시각	품질판정	점수	모델버전
STK-20260619-EF56	박집사	2026-06-19T01:25:10.000Z	재활영 권장	32	v1
STK-20260619-CD34	이랫	2026-06-19T01:14:30.000Z	보정 권장	58	v1
STK-20260619-AB12	김보호	2026-06-19T01:11:00.000Z	제작 적합	82	v1

운영 로그 요약 (예측 · 피드백)

예측 요청

4건

평균 점수

58.19

피드백

1건

정정 건수

1

5. Git 기반 개발 과정

- 개발 흐름: main 보호, feature/mlops-fastapi-pipeline 브랜치에서 기능 단위로 개발 후 PR.
- 커밋 전략: Conventional Commits(feat / ci / docs) + 한글 본문, 기능 단위로 8개 이상 분리.
 - feat(ml): 스티커 품질 ML 패키지 골격과 특징 추출 추가
 - feat(ml): MLflow 학습 파이프라인과 챔피언 내보내기 구현

- o feat(ml): FastAPI 추론 서비스와 예측·피드백 로깅 추가
- o feat(next): 주문 API와 ML 품질 결과 연동
- o feat(next): 관리자 페이지로 주문·모델·로그 확인
- o feat(ops): Docker·compose·재학습/롤백 CLI·외부배포 설정 추가
- o ci: 통합 하네스·자동학습 워크플로 구성
- o docs: 최종 보고서 작성 및 실행 캡처 삽입
- 브랜치 사용 여부: 사용. 기능마다 feature/* 브랜치 → PR → 머지. 기존 사용자 변경/원격 브랜치는 보존.
- 지속적 개발(PR 이력): 초기 파이프라인(PR #3) → 배포 확인(#4) 이후, 실서비스 강화 기능을 각각 별도 PR 로 추가·머지했다 — 관리자 인증(#5), 인앱 피드백(#6), 주문 상태 관리(#7), 모니터링·재학습 트리거(#8), 재학습 라이브 검증(#9). 모든 PR 이 CI(next·ml·harness·docker) 통과 후 머지(18절 참조).

6. CI/CD 구성

.github/workflows/ci.yml — push/PR 트리거, 4개 잡:

- next: npm ci → typecheck → build
- ml: setup-python 3.12 → ruff → pytest → MLflow 학습 → 학습 artifact 업로드(champion/, mlruns/)
- harness: Node+Python 설치 후 HARNESS_STRICT=1 node scripts/final-harness.mjs (앱↔ML 교차검증)
- docker(push 한정): ML/Next 이미지 빌드

.github/workflows/auto-train.yml — 자동 재학습: pet-sticker-ml/data/** · train.py · dataset.py 변경 또는 workflow_dispatch 시 pytest → train → model_promoter(승격 평가) → artifact 업로드.

The screenshot shows the GitHub Actions interface for the repository **thighburger / StickerGenerator**. The workflow **fix(next): 주문 API 버퍼를 BlobPart 호환 타입으로 수정 #6** is displayed, triggered via push 14 minutes ago. The status is **Success**, with a total duration of **2m 20s** and **1** artifact.

The workflow file **ci.yml** is shown with the trigger **on: push**. The jobs are visualized as follows:

- next** (44s) and **ml** (59s) run in parallel.
- harness** (1m 17s) runs after the parallel jobs.
- docker** (1m 34s) runs separately.

The **Annotations** section shows 4 warnings, including a deprecation warning for Node.js 20.

7. Docker 기반 환경 구성

- ML(`pet-sticker-ml/Dockerfile`): `python:3.12-slim` + `libgomp1` , 의존성 설치, `src/champion/data` 복사, 비루트 실행, `HEALTHCHECK /health` , `uvicorn ... --factory` (포트 8000).
- Next(`pet-sticker-next/Dockerfile`): `deps`→`builder`→`runner` 멀티스테이지(`node:20-alpine`), `sample-orders` 포함.
- `docker-compose.yml` : `ml` (8000, healthcheck, logs 볼륨) + `next` (3000, `depends_on: ml healthy` , `ML_SERVICE_URL=http://ml:8000`) + `mlflow` (profile `tracking` , 5000).
- 실행: `docker compose up -d --build` → `curl localhost:8000/health` → `localhost:3000` .

Docker 실행 상태(`docker compose ps`)는 로컬에서 확인하며, CI 의 `docker` 잡에서 ML/Next 두 이미지 빌드가 통과했다(6절 GitHub Actions 캡처 참조).

8. ML 모델 구성

- 사용 데이터: 라벨링된 공개 데이터가 없어, 특징값을 현실적 분포에서 샘플링하고 문서화된 품질 점수 휴리스틱 + 라벨 노이즈로 3분류 라벨을 만든 합성 데이터(시드 고정, 완전 재현). 추론 시에는 실제 업로드 이미지에서 동일한 특징 10종(해상도·메가픽셀·종횡비·밝기·대비·선명도(Laplacian 분산)·색감·피사체 비율·엣지밀도)을 PIL/OpenCV 로 추출 → 진짜 이미지 ↔ ML 연결.
- 모델 종류: `Pipeline(StandardScaler → RandomForestClassifier(class_weight=balanced))` , 3분류. 품질 점수 (0~100)는 클래스 확률 가중합($100 \cdot P(\text{적합}) + 50 \cdot P(\text{보정})$)으로 산출.
- 학습 코드: `train.py` — train/test 분할 → 학습 → 평가 → MLflow 기록 → 레지스트리 등록 → 챔피언 export.
- 평가 지표: accuracy, macro-F1(승격 기준), 클래스별 F1, score_mae.
- 초기(v1) vs 신규(v2) 비교:

모델	데이터	표본	macro-F1	accuracy
v1 (초기)	sticker_quality_v1	1500	0.7549	0.7567
v2 (재학습)	sticker_quality_v2	2200	0.7813	0.78+

v2 는 표본 증가 + 라벨 노이즈 감소로 macro-F1 이 향상되어 챔피언으로 승격된다(라이브 검증 결과는 18-4절).

9. MLflow 기반 실험 관리

- Tracking 사용: 로컬 파일스토어(`file:./mlruns`)를 기본으로, 보고서 캡처는 `mlflow server (sqlite, :5000)`.
- 기록 항목:
 - parameter: `data_version`, `seed`, `n_estimators`, `test_size`, `model_type`, `feature_count`, `n_samples` 등
 - metric: `accuracy`, `macro_f1`, `score_mae`, 클래스별 f1
 - artifact: `features.json` , `confusion_matrix.png` , `classification_report.txt` , `dataset_version.txt` , 입력 CSV
 - model: `mlflow.sklearn.log_model` + 레지스트리 `pet-sticker-quality` 등록
- 최고 모델 선정 기준: macro-F1 이 현재 챔피언 + `min-delta` 이상 개선되면 `@champion` 승격.

The screenshot shows the MLflow Experiments page for an experiment named 'pet-sticker-quality'. The interface includes a sidebar with the experiment name, a search bar, and a list of experiments. The main panel shows the 'Runs' tab with a table of runs. The table has columns for Run Name, Created, Dataset, Duration, Source, and Models. There are 5 runs listed, all with a status of 'Active' and a 'Created' time of '1 hour ago' or '22 seconds ago'. The runs are named 'quality-rf-v1' and 'quality-rf-v2'.

Run Name	Created	Dataset	Duration	Source	Models
quality-rf-v2	22 seconds ago	-	3.0s	train.py	pet-sticker-quality
quality-rf-v1	1 minute ago	-	38.0s	train.py	pet-sticker-quality
quality-rf-v1	1 hour ago	-	2.1s	train.py	pet-sticker-quality
quality-rf-v2	1 hour ago	-	13.8s	retrain.py	pet-sticker-quality
quality-rf-v1	1 hour ago	-	35.3s	train.py	pet-sticker-quality

10. 모델 등록 및 서비스 반영

- 저장 방식: MLflow 레지스트리(`pet-sticker-quality` , `@champion` alias) + 포터블 `export champion/model.pkl + champion/metadata.json` (버전: `metric.featureNames.classes`).
- 서비스 로드: FastAPI 가 시작 시 `champion/` 만 로드(서버 불필요 → CI/Docker 재현성). `GET /model/info` 로 버전 확인.
- 신규 반영: 승격 후 `POST /model/reload` 로 무중단 재로딩(또는 컨테이너 재시작).
- 자동/수동 선택: 반자동 — 학습은 자동(CI), 승격은 `model_promoter` 가 metric 비교로 자동 결정하되, 운영 반영은 명시적 reload/ 배포로 통제(안전성 우선). 근거: 잘못된 모델의 즉시 무중단 반영 리스크 차단.

Pretty-print ☐

```
{
  "modelName": "pet-sticker-quality",
  "alias": "champion",
  "version": 1,
  "runId": "408960c1334e4588aec177291f79d1a8",
  "dataVersion": "v1",
  "featureNames": [
    "width",
    "height",
    "megapixels",
    "aspect_ratio",
    "brightness",
    "contrast",
    "sharpness",
    "colorfulness",
    "subject_ratio",
    "edge_density"
  ],
  "classes": [
    "제작 적합",
    "보정 권장",
    "재촬영 권장"
  ],
  "scoreThresholds": {
    "pass": 70.0,
    "retouch": 45.0
  },
  "metrics": {
    "accuracy": 0.7567,
    "macro_f1": 0.7549,
    "score_mae": 15.1316,
    "f1_제작 적합": 0.7737,
    "f1_보정 권장": 0.7638,
    "f1_재촬영 권장": 0.7273
  },
  "params": {
    "data_version": "v1",
    "seed": 42,
    "n_estimators": 200,
    "test_size": 0.2,
    "model_type": "RandomForestClassifier",
    "feature_count": 10,
    "n_samples": 1500,
    "n_train": 1200,
    "n_test": 300,
    "trainedAt": "2026-06-19T16:26:56.005227+00:00",
    "modelVersion": "v1"
  }
}
```

11. 재학습 또는 모델 개선 과정

- **이유/방법:** 사용자 피드백(흐릿/경계 사례 오판)을 반영해 **라벨 노이즈를 줄이고 표본을 늘린 데이터 v2** 로 `python -m pet_sticker_ml.retrain --data-version v2` 재학습.
- **무엇이 바뀌었나:** 데이터(v1→v2). 코드/파라미터는 동일.
- **전/후 성능:** macro-F1 0.7549 → **0.7783** 로 향상 → `model_promoter` 가 v2 를 챔피언(v2)으로 승격.
- **모델 교체 결과:** 이전 챔피언(v1)은 `model_history/v1/` 에 보관, 서비스는 v2 로 전환.

12. 운영 로그 및 문제 대응

- **서비스 로그:** 컨테이너 stdout + Next API 응답.
- **예측 요청 로그:** `logs/prediction_log.csv` (시간·요청ID·소스·모델버전·점수·클래스·신뢰도+특징값).
- **피드백 로그:** `logs/feedback_log.csv` (예측/교정 클래스 → 재학습 데이터).
- **모델 정보 확인:** `GET /model/info` , 관리자 화면.
- **일부러 발생시킨 문제 1:** 로컬 Python 3.14 환경에서 `mlflow / scikit-learn` 휠 부재로 설치 실패.
 - 원인: 3.14 가 너무 최신이라 바이너리 휠 미배포.
 - 해결: ML 환경을 **3.12 로 고정**(`uv` 로 standalone CPython 설치, `.python-version` , CI `setup-python 3.12`), 하네스는 3.11/3.12 자동 탐색 + 미존재 시 명확한 SKIP 안내.
- **문제 2:** `mlflow.sklearn.log_model(name=...)` 인자 오류(2.x 는 `artifact_path`). → 시그니처 수정으로 모델/레지스트리 정상 기록.

Pretty-print ☐

```
{"status": "ok", "modelLoaded": true, "modelVersion": "v1"}
```

Pretty-print ☐

```
{
  "prediction_count": 5,
  "class_counts": {
    "보정 권장": 2,
    "제작 적합": 2,
    "재촬영 권장": 1
  },
  "mean_score": 54.7,
  "feedback_count": 0,
  "correction_count": 0,
  "correction_rate": null,
  "recent_predictions": [
    {
      "timestamp": "2026-06-19T17:26:01.485042+00:00",
      "request_id": "e5c8b3d33840",
      "source": "features",
      "model_version": "v1",
      "score": 98.25,
      "quality_class": "제작 적합",
      "confidence": 0.965,
      "width": 2048,
      "height": 2048,
      "megapixels": 4.19,
      "aspect_ratio": 1.0,
      "brightness": 142,
      "contrast": 49,
      "sharpness": 580,
      "colorfulness": 41,
      "subject_ratio": 0.55,
      "edge_density": 0.16
    },
    {
      "timestamp": "2026-06-19T17:26:01.462761+00:00",
      "request_id": "ea8f4f13e93f",
      "source": "features",
      "model_version": "v1",
      "score": 2.0,
      "quality_class": "재촬영 권장",
      "confidence": 0.96,
      "width": 800,
      "height": 600,
      "megapixels": 0.48,
      "aspect_ratio": 1.33,
      "brightness": 58,
      "contrast": 18,
      "sharpness": 72,
      "colorfulness": 12,
      "subject_ratio": 0.22,
      "edge_density": 0.04
    },
    {
      "timestamp": "2026-06-19T17:26:01.441413+00:00",
      "request_id": "ead7be35aldd",
      "source": "features",
      "model_version": "v1",
      "score": 34.25,
      "quality_class": "보정 권장",
      "confidence": 0.655,
      "width": 1600,
      "height": 1200,
      "megapixels": 1.92,
      "aspect_ratio": 1.33,
      "brightness": 96,
      "contrast": 33,
      "sharpness": 220,
      "colorfulness": 24,
      "subject_ratio": 0.41,
      "edge_density": 0.09
    },
    {
      "timestamp": "2026-06-19T17:26:01.409936+00:00",
      "request_id": "69179f6fdffd",
      "source": "features",
      "model_version": "v1",
      "score": 99.5,
      "quality_class": "제작 적합",
      "confidence": 0.99,
      "width": 2400,
      "height": 1800,
      "megapixels": 4.32,
      "aspect_ratio": 1.33,
      "brightness": 138,
      "contrast": 52,
      "sharpness": 640,
      "colorfulness": 38,
      "subject_ratio": 0.52,
      "edge_density": 0.18
    },
    {
      "timestamp": "2026-06-19T16:26:59.397610+00:00",
      "request_id": "6fb7426f2798",
      "source": "features",
      "model_version": "v1",
      "score": 39.5,
      "quality_class": "보정 권장",
      "confidence": 0.75,
      "width": 1200,
      "height": 1000,
      "megapixels": 1.2,
      "aspect_ratio": 1.2,
      "brightness": 130,
      "contrast": 45,
      "sharpness": 300,
      "colorfulness": 30,
      "subject_ratio": 0.5,
      "edge_density": 0.1
    }
  ],
  "recent_feedback": []
}
```

13. 롤백 및 이전 모델 관리

- 이전 모델 보관: 승격 시 직전 챔피언을 `model_history/v{n}/` 에 자동 보관.
- 롤백 방법: `python -m pet_sticker_ml.rollback --list` 로 버전 확인 → `python -m pet_sticker_ml.rollback --to 1` 로 v1 복원(현재 챔피언도 보관 후 교체, 레지스트리 alias 동기화).
- 버전 관리: `champion/metadata.json` 의 `version / alias / metrics / runId` + MLflow 레지스트리 `@champion` .
- 실제 동작: v2 승격 → 롤백 `--to 1` → 챔피언이 v1(`macro_f1 0.7549, rolledBack=true`)로 복귀 확인.

14. 전체 파이프라인 동작 흐름

- 코드 수정 → 서비스 반영: 커밋/푸시 → CI(next/ml/harness) 통과 → Vercel/Render 배포.
- 데이터 변경 → 재학습: `data/**` 변경 푸시 → `auto-train.yml` 자동 실행(test→train→promote→artifact).
- 모델 변경 → 운영 반영: 승격으로 `champion/` 갱신 → `/model/reload` → `/model/info` 버전 변경 확인.

15. 문제 해결 경험

- 12절의 두 문제(3.14 휠, log_model 시그니처) 외에:
- 하네스 플레이키 방지: 포트 바인딩 의존을 없애기 위해 예측/로그 검증은 순수 함수 호출, 모델정보 API 는 FastAPI `TestClient`(인프로세스) 로 검증. 실제 포트 헬스체크는 `--full` 모드의 docker compose 단계로 분리.
- `.gitignore` 충돌: `orders/` 규칙이 `app/api/orders/` 라우트까지 무시 → `/orders/` 로 한정해 해결.

16. 느낀 점 및 개선 방향

- **배운 점:** 모델 정확도보다 학습→등록→서빙→교체→롤백→모니터링→재학습의 연결과 재현성이 MLOps 의 핵심임을 체감. champion 포터블 export 로 서빙을 MLflow 서버와 분리하니 CI/Docker 재현이 쉬워졌다.
- **개선하고 싶은 점:** 실제 라벨 데이터 수집·라벨링 파이프라인, 데이터/모델 드리프트 모니터링, 자동 승격→배포까지 무중단 연결.
- **수업 피드백:** MLflow·GitHub Actions 자동 훈련·모델 교체/롤백·피드백 모니터링 등 강의 내용이 실제 프로젝트에 그대로 적용되어 큰 도움이 되었다. 실습에 경량 합성 데이터로 빠르게 전체 루프(학습→배포→재학습)를 도는 예제가 추가되면 MLOps 흐름을 더 빠르게 체득할 수 있을 것 같다.

17. 참고 자료

- 강의 자료: MLOps 개요, MLflow 로컬/서버 실습, GitHub Actions 기반 자동 훈련, 모델 교체/운영 전략, 사용자 피드백·모니터링, 로깅, DevOps 정리
- MLflow / scikit-learn / FastAPI / Next.js 공식 문서
- 중간 프로젝트 보고서(스티커 생성 서비스 MVP)

18. 실서비스 강화 기능 (추가 구현)

초기 파이프라인 완성 후, **실제 서비스로서 부족한 점**을 점검해 기능을 추가했다. 각 기능은 별도 PR 로 구현·리뷰·머지했다(기능 단위 커밋·CI 통과·스크린샷 포함).

18-1. 관리자 인증 (보안)

문제(부족한 점): 운영 관리자 페이지 `/admin` 이 인증 없이 공개되어, 누구나 전체 고객의 이름·연락처·주소가 담긴 주문 정보를 열람할 수 있었다(개인정보 노출 위험).

구현:

- Next.js **미들웨어**(`middleware.ts`)로 `/admin/:path*` 전 경로를 보호. 유효 세션 쿠키가 없으면 `/admin/login` 으로 리다이렉트(`GET /admin` → `307`).
- 로그인 페이지(`/admin/login`)에서 비밀번호 확인 → `ADMIN_SECRET` 기반 **HMAC-SHA256 세션 토큰**을 httpOnly 쿠키로 발급 (`lib/auth.ts` , Web Crypto 로 Edge/Node 공용). 비밀번호는 쿠키/클라이언트에 노출되지 않음.
- 대시보드에 **로그아웃** 버튼 추가. 비밀번호·시크릿은 환경변수(`ADMIN_PASSWORD` , `ADMIN_SECRET`)로 주입.

운영 관리자 로그인

주문·모델 로그 대시보드는 관리자 전용입니다.

관리자 비밀번호

로그인

로그인 성공 후에는 위 4절의 관리자 대시보드(라이브 챔피언 모델·주문 통계·로그 요약)로 진입한다.

18-2. 인앱 ML 피드백 (human-in-the-loop)

문제(부족한 점): ML 품질 판정에 대한 사용자 의견을 수집할 화면이 없어, `/api/feedback` (및 ML `/feedback`)가 있어도 피드백이 실제로 쌓이지 않았다(재학습 데이터 공백).

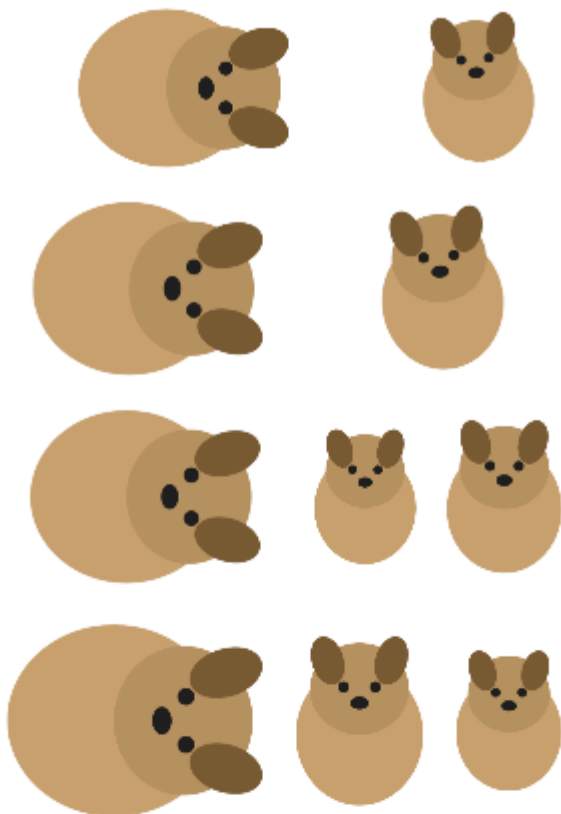
구현:

- 구매 후 표시되는 **품질 결과 카드**에 피드백 위젯(`MLFeedback`) 추가: "👍 정확해요" 또는 "✎ 다른 판정"(올바른 클래스 선택) → `/api/feedback` → ML `/feedback` → **피드백 로그** 적재.
- 예측 `requestId` ·예측 클래스·교정 클래스·주문번호를 함께 기록해 **재학습 데이터**로 활용.
- 실패 시에도 사용자 흐름을 막지 않으며(*graceful*), 제출 후 감사 메시지를 표시한다.

아래는 실제 구매 흐름에서 **라이브 Render 모델(v1)** 이 "재촬영 권장(24점)"으로 판정하고, 사용자가 그 판정을 확인/교정할 수 있는 화면이다.

실시간 시안 미리보기

A6 스티커 1장



-

100%

+

원본 크기

A6 스티커 1장

~~₩~~9,900

배송비 ₩2,500 · 3만원 이상 무료

수량 선택

-

1

+

총 결제금액

~~₩~~9,900

최영현

010-1234-5678

서울특별시 마포구 명령로 12

요청사항

구매하기

STK-20260621-0GST 주문번호가 생성됐어요. 팝업이 막
히면 카카오톡 문의 버튼을 다시 눌러주세요.

재촬영 권장

24점

해상도가 낮거나 흐릿/어두워 인쇄 품질이 떨어질 수 있습니다.
재촬영을 권장합니다. 피사체 크기가 적절하지 않습니다. 화면
을 적당히 채우도록 촬영하세요.

모델 v1 신뢰도 53%

이 품질 판정이 정확한가요?

👍 정확해요

✎ 다른 판정

18-3. 주문 상태 관리 + 저장소 추상화

문제(부족한 점): 주문에 상태(접수/제작/배송) 개념이 없어 운영자가 진행 상황을 관리할 수 없었고, 주문이 로컬 파일(`orders/`)에만 저장되어 서버 재배포 시 유실되었다.

구현:

- 주문 상태 lifecycle 도입: 접수됨 → 결제확인 → 제작중 → 배송중 → 배송완료(신규 주문은 접수됨).
- 관리자 대시보드 주문 목록에 **주문상태 컬럼 + 변경 선택**(`OrderStatusControl`) 추가. 변경 시 `/api/admin/orders/[orderId]/status` 호출 → 파일에 영속. 해당 API 도 **미들웨어로 보호**(미인증 401).
- 저장소 추상화**(`lib/order-store.ts`)로 읽기/목록/상태변경/저장을 한 곳에 모아 추후 DB(Supabase/ Postgres)로 교체해도 호출 부 변경이 없게 했다. (파일 기반은 서버리스에서 휘발성이므로 운영 영속성은 외부 DB/Render 영속 디스크로 확장 권장 — 추상화로 교체 비용 최소화.)

아래는 실제 주문을 생성하고 관리자가 상태를 "제작중"으로 변경한 화면이다.

운영 관리자 대시보드

주문별 ML 품질 점수 · 챔피언 모델 버전 · 예측/피드백 로그

← 생성기로

로그아웃

챔피언 모델

모델 pet-sticker-quality	버전 v1	데이터 v1	macro-F1 0.7549	정확도 0.7567	학습 시각 2026-06-19T16:26:56.005227+00:00
----------------------------------	-----------------	------------------	---------------------------	----------------------	--

1

주문 수

24

평균 품질점수

0

제작 적합

0

보정 권장

1

재촬영 권장

주문 목록

주문번호	이름	생성시각	품질판정	점수	모델버전	주문상태
STK-20260621-0TWO	최영현	2026-06-21T02:53:42.474Z	재촬영 권장	24	v1	제작중 ▼

운영 로그 요약 (예측 · 피드백)

예측 요청 6건	평균 점수 46.79	피드백 1건	정정 건수 1
--------------------	-----------------------	------------------	-------------------

18-4. ML 모니터링 대시보드 + 재학습 트리거

문제(부족한 점): 관리자 화면이 숫자 위주라 운영 추이를 한눈에 보기 어려웠고, 재학습이 CLI 로만 가능했다(운영 중 모델 개선이 번거로움).

구현:

- 관리자 대시보드에 **품질 판정 분포 막대 차트**(예측 로그 기반)와 평균 품질점수·챔피언 버전 패널 추가.
- 재학습 트리거 버튼**: `/api/admin/retrain` (미인증 401) → ML 서비스 `POST /admin/retrain` → 데이터 v2 재학습 → macro-F1 개선 시 **챔피언 승격** → 챔피언 reload. 결과(승격 여부·버전·지표)를 화면에 표시.
- 운영자가 **버튼 클릭만으로 재학습→평가→배포 반영** 루프를 실행할 수 있다(배포된 Render ML 에 반영).

아래는 라이브 Render 예측 로그(11건) 기반 품질 분포 차트와 재학습 트리거 버튼이다.

운영 관리자 대시보드

주문별 ML 품질 점수 · 챔피언 모델 버전 · 예측/피드백 로그

← 생성기로

로그아웃

챔피언 모델

모델	버전	데이터	macro-F1	정확도	학습 시각
pet-sticker-quality	v1	v1	0.7549	0.7567	2026-06-19T16:26:56.005227+00:00

1

주문 수

24

평균 품질점수

0

제작 적합

0

보정 권장

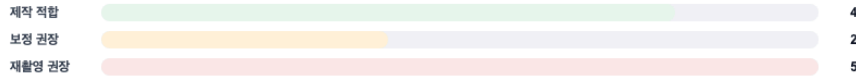
1

재촬영 권장

ML 모니터링 · 재학습

🔄 재학습 트리거

품질 판정 분포 (11건)



평균 품질점수
47.14
챔피언 v1

주문 목록

주문번호	이름	생성시각	품질판정	점수	모델버전	주문상태
STK-20260621-0TW0	최영현	2026-06-21T02:53:42.474Z	재촬영 권장	24	v1	제작중 ▼

운영 로그 요약 (예측 · 피드백)

예측 요청 11건	평균 점수 47.14	피드백 1건	정정 건수 1
--------------	----------------	-----------	------------

라이브 검증(배포된 Render ML 에 실제 재학습 트리거): v1 → v2 승격이 운영 서비스에 반영됨.

```
# 재학습 전: champion v1, macro_f1 0.7549 (dataVersion v1)
$ curl -X POST https://pet-sticker-ml.onrender.com/admin/retrain
{"promoted":true,"previous_macro_f1":0.7549,"new_macro_f1":0.7813,"champion_version":2,"dataVersion":"v2"}
# 재학습 후: GET /model/info → version:2, dataVersion:v2, macro_f1:0.7813
```

즉 재학습 → 평가 → 챔피언 승격 → 서비스 반영(/model/info) 전체 루프가 배포 환경에서 동작함을 확인했다.

부록 A. 평가 기준 ↔ 구현 대응표

평가 기준(배점)	대응 구현
MLOps 파이프라인 완성도(35)	Git→CI(ci.yml)→Docker(compose)→MLflow(train.py)→배포(render.yaml /Vercel) 전체 연결, 하네스로 교차검증
Git 이력/개발 과정(10)	feature 브랜치 + 한글 기능단위 커밋 8+개 + PR
자동화(10)	push 시 typecheck/build/test 자동, auto-train.yml 자동 재학습·승격
MLflow 활용/모델 관리(15)	experiment/run, param·metric·artifact·model 기록, 레지스트리+@champion, v1/v2 비교
앱·ML 기능(10)	/api/orders →ML /predict , 결과 카드·주문 저장·관리자 화면 연동
Docker/실행환경(5)	ML/Next Dockerfile + compose + healthcheck
배포/운영(5)	Vercel/Render 배포, /health ·예측/피드백 로그·관리자 모니터링
보너스(10)	롤백 CLI, 자동 승격, 강화된 테스트(23개)+통합 하네스, 운영 로그 분석, 재학습 파이프라인, 관리자 인증(보안) 등 실서비스 강화(18절)

부록 B. 캡처 목록 및 작성 환경 안내

docs/assets/ 에 포함된 실제 캡처:

파일	내용
mlflow-ui.png	MLflow 실험 run 목록(5 run, train.py/retrain.py 출처, 등록 모델)
model-info.png	챔피언 모델 정보 API(버전·alias·metric·featureNames)
logs-summary.png	예측/피드백 로그 요약(운영 로그)
fastapi-docs.png	FastAPI 추론 서비스 Swagger 문서
health.png	서비스 상태(/health , 로컬)
github-actions.png	CI 전체 잡 통과(next·ml·docker·harness) + workflow 그래프
deploy-render-health.png	라이브 Render 배포 /health 외부 응답
deploy-render-docs.png	라이브 Render 배포 FastAPI Swagger(/docs)
app-main.png	웹 앱 메인(업로드·시안 생성·구매)
admin-login.png	관리자 로그인 화면(보안)
admin-dashboard.png	관리자 대시보드(로그인 후, 라이브 챔피언·주문·로그)
ml-feedback.png	인앱 ML 피드백 위젯(품질 결과 카드, 라이브 모델 판정)
admin-order-status.png	관리자 주문 상태 관리(상태 lifecycle 선택)
admin-monitoring.png	ML 모니터링 대시보드(품질 분포 차트 + 재학습 트리거)

실제 외부 배포 완료: ML 서비스는 Render 에 라이브 배포되어 <https://pet-sticker-ml.onrender.com> 으로 외부에서 접속·동작이 확인된다. 웹 앱(Next.js)도 Vercel `main` 에 자동 배포되어 빌드 성공 상태다.

캡처는 모두 자동화 스크립트로 생성한다(로컬에서 앱을 띄우고 Playwright 로 촬영):

```
npm run dev    # 또는 docker compose up -d --build
npm run capture
```

참고: Vercel 배포에는 배포 보호(인증)가 켜져 있어 배포된 URL 의 외부 자동 캡처는 401 로 차단되므로, 앱/관리자 화면은 로컬 실행본을 라이브 Render ML 에 연결해 촬영했다(데이터·동작은 동일).